S T A N D A R D O P E R A T I N G P R O C E D U R E

BYUH Admissions RAG Chatbot

Building an AI-Powered Admissions Assistant from Scratch

Next.js · PostgreSQL + pgvector · Vercel AI SDK · Claude Code

Version	1.0
Date	March 18, 2026
Author	InnovatorsDNA Team
Classification	Internal

Brigham Young University – Hawaii | admissions.byuh.edu

1. Overview

This SOP walks you through building a production-ready RAG (Retrieval-Augmented Generation) chatbot for BYU–Hawaii admissions. By the end, you will have a fully functional AI assistant that answers prospective student questions using real content scraped from admissions.byuh.edu.

What is RAG?

Retrieval-Augmented Generation is a technique where you:

- Scrape and store content from a website
- Split that content into small chunks and generate vector embeddings
- When a user asks a question, find the most relevant chunks via vector similarity search
- Feed those chunks as context to an LLM, which generates a grounded answer

Total build time: ~1 hour following this SOP step-by-step

2. Tech Stack

Every technology used in this project and why it was chosen:

Category	Technology	Why
Framework	Next.js 16 (App Router)	Full-stack React with API routes, SSR, serverless deploy
Language	TypeScript	Type safety catches bugs before runtime
Database	Neon PostgreSQL	Serverless Postgres with built-in pgvector support
Vector Search	pgvector	Cosine similarity search directly in Postgres
ORM	Drizzle ORM	Type-safe SQL, lightweight, great migration tooling
AI SDK	Vercel AI SDK v6	Streaming responses, useChat hook, AI Gateway
Embeddings	text-embedding-3-small	1536 dimensions, fast, cheap, excellent quality
LLM	GPT-4o-mini	Fast, cheap, good instruction following
Scraping	Axios + Cheerio	Simple HTTP + DOM parsing, no browser needed
Styling	Tailwind CSS v4	Utility-first CSS, rapid UI development
Pkg Manager	pnpm	Fast, disk-efficient
AI Tool	Claude Code (CLI)	AI pair programmer in your terminal

3. Prerequisites

Before you begin, make sure you have the following installed and ready.

3.1 Software Requirements

Tool	Version	Check Command
Node.js	v22 or later	node --version
pnpm	v9+	pnpm --version
Git	Any recent version	git --version
Claude Code CLI	Latest	claude --version

Install pnpm if you don't have it:

```
npm install -g pnpm
```

Install Claude Code if you don't have it:

```
npm install -g @anthropic-ai/claude-code
```

3.2 Accounts & API Keys

You need accounts on these services (all have free tiers):

- **Neon (neon.tech)** — Create a free PostgreSQL database. Copy the connection string.
- **Vercel (vercel.com)** — For the AI Gateway key. Dashboard > AI > Gateway.
- **GitHub (github.com)** — For version control and deployment.

NOTE: *Never commit API keys to git. Always use a .env file and make sure .env is in your .gitignore.*

4. Phase 1 — Project Setup & Database

Create the Next.js project, install Drizzle ORM, connect to Neon, and set up the vector database schema.

4.1 Create the Next.js Project

Open your terminal and run:

```
pnpm create next-app@latest byuh-admissions-bot

# When prompted, select:
#   TypeScript:     Yes
#   ESLint:         Yes
#   Tailwind CSS:   Yes
#   src/ directory:  Yes
#   App Router:     Yes
#   Import alias:   @/*
```

Navigate into the project:

```
cd byuh-admissions-bot
```

4.2 Set Up Environment Variables

Create a .env file in the project root:

```
# .env
DATABASE_URL='postgresql://user:pass@host/db?sslmode=require'
AI_GATEWAY_API_KEY='your-vercel-ai-gateway-key'
```

NOTE: *Replace the placeholder values with your actual Neon connection string and Vercel AI Gateway key.*

4.3 Install All Dependencies

Run these commands to install every package:

```
# Core AI + database
pnpm add ai @ai-sdk/gateway @ai-sdk/openai @ai-sdk/react

# Database
pnpm add drizzle-orm pg dotenv

# Web scraping
pnpm add axios cheerio

# UI enhancements
pnpm add react-markdown remark-gfm lucide-react
pnpm add @tailwindcss/typography

# Concurrency control
pnpm add p-limit

# Dev dependencies
```

```
pnpm add -D drizzle-kit tsx @types/pg
```

4.4 Set Up the Database Connection

Set up Drizzle ORM with our Neon PostgreSQL database. Use the pg driver (not @neondatabase/serverless) and load env from dotenv. Put the files in src/db/.

Create src/db/index.ts — Database connection:

```
import { drizzle } from 'drizzle-orm/node-postgres';
import { Pool } from 'pg';
import * as schema from './schema';
import { config } from 'dotenv';
config({ path: '.env' });

const pool = new Pool({
  connectionString: process.env.DATABASE_URL!
});
export const db = drizzle(pool, { schema });
export { pool };
```

Create drizzle.config.ts — in the project root:

```
import { type Config } from 'drizzle-kit';
import { config } from 'dotenv';
config({ path: '.env' });

export default {
  schema: './src/db/schema.ts',
  dialect: 'postgresql',
  dbCredentials: {
    url: process.env.DATABASE_URL!
  },
  tablesFilter: ['byuh_*'],
} satisfies Config;
```

4.5 Create the Database Schema

Build a schema for a RAG chatbot. We need a chunks table with pgvector embeddings (1536 dimensions for OpenAI text-embedding-3-small), plus tables for session management and conversation history.

Create src/db/schema.ts — All table definitions:

```
import {
  customType, integer, serial, text,
  timestamp, uuid
} from 'drizzle-orm/pg-core';
import { pgTable } from 'drizzle-orm/pg-core';

// Custom pgvector type (1536 dims = OpenAI
// text-embedding-3-small)
const vector = customType<{
  data: number[];
  driverData: string
}>({
  dataType() { return 'vector(1536)'; },
  fromDriver(value: string): number[] {
    return value
      .slice(1, -1).split(',').map(Number);
  },
  toDriver(value: number[]): string {
    return '[' + value.join(',') + ']';
  },
});

// RAG chunks with embeddings
export const byuhChunks = pgTable(
  'byuh_chunks', {
  id: serial('id').primaryKey(),
  url: text('url').notNull(),
  title: text('title').notNull().default(''),
  content: text('content').notNull(),
  chunkIndex: integer('chunk_index').notNull(),
  embedding: vector('embedding'),
  createdAt: timestamp('created_at')
    .defaultNow(),
});

// Session tracking (anonymous visitors)
export const sessions = pgTable('sessions', {
  id: uuid('id').primaryKey().defaultRandom(),
  createdAt: timestamp('created_at')
    .defaultNow(),
});

// Conversation threads
```

```
export const conversations = pgTable(
  'conversations', {
  id: uuid('id').primaryKey().defaultRandom(),
  sessionId: uuid('session_id').notNull()
    .references(() => sessions.id),
  title: text('title').notNull()
    .default('New conversation'),
  createdAt: timestamp('created_at')
    .defaultNow(),
});

// Individual chat messages
export const chatMessages = pgTable(
  'chat_messages', {
  id: serial('id').primaryKey(),
  conversationId: uuid('conversation_id')
    .notNull()
    .references(() => conversations.id,
      { onDelete: 'cascade' }),
  role: text('role').notNull(),
  content: text('content').notNull(),
  createdAt: timestamp('created_at')
    .defaultNow(),
});
```

4.6 Create the Migration Script

We use a direct SQL migration script (avoids Drizzle Kit issues with pgvector indexes):

Create src/db/migrate.ts:

```
import { Pool } from 'pg';
import { config } from 'dotenv';
config({ path: '.env' });

async function migrate() {
  const pool = new Pool({
    connectionString: process.env.DATABASE_URL!
  });
  const client = await pool.connect();
  try {
    await client.query(
      'CREATE EXTENSION IF NOT EXISTS vector;'
    );

    await client.query(`
      CREATE TABLE IF NOT EXISTS byuh_chunks (
        id SERIAL PRIMARY KEY,
        url TEXT NOT NULL,
        title TEXT NOT NULL DEFAULT '',
        content TEXT NOT NULL,
```

```
      chunk_index INTEGER NOT NULL,
      embedding vector(1536),
      created_at TIMESTAMPTZ DEFAULT NOW()
    );
  `);

  await client.query(`
    CREATE INDEX IF NOT EXISTS
      byuh_chunks_embedding_idx
    ON byuh_chunks
    USING ivfflat (embedding vector_cosine_ops)
    WITH (lists = 100);
  `);

  await client.query(`
    CREATE TABLE IF NOT EXISTS sessions (
      id UUID PRIMARY KEY
        DEFAULT gen_random_uuid(),
      created_at TIMESTAMPTZ DEFAULT NOW()
    );
  `);

  await client.query(`
    CREATE TABLE IF NOT EXISTS conversations (
      id UUID PRIMARY KEY
        DEFAULT gen_random_uuid(),
      session_id UUID NOT NULL
        REFERENCES sessions(id),
      title TEXT NOT NULL
        DEFAULT 'New conversation',
      created_at TIMESTAMPTZ DEFAULT NOW()
    );
  `);

  await client.query(`
    CREATE TABLE IF NOT EXISTS chat_messages (
      id SERIAL PRIMARY KEY,
      conversation_id UUID NOT NULL
        REFERENCES conversations(id)
        ON DELETE CASCADE,
      role TEXT NOT NULL,
      content TEXT NOT NULL,
      created_at TIMESTAMPTZ DEFAULT NOW()
    );
  `);

  await client.query(`
    CREATE INDEX IF NOT EXISTS
      chat_messages_conversation_idx
    ON chat_messages
```

```
        (conversation_id, created_at);
    `);

    console.log('All migrations complete!');
  } finally {
    client.release();
    await pool.end();
  }
}

migrate().catch((err) => {
  console.error('Migration failed:', err);
  process.exit(1);
});
```

4.7 Add Scripts to package.json

```
"scripts": {
  "dev": "next dev",
  "build": "next build",
  "start": "next start",
  "lint": "eslint",
  "db:migrate": "tsx src/db/migrate.ts",
  "ingest": "tsx src/scripts/ingest.ts"
}
```

4.8 Run the Migration

```
pnpm db:migrate
```

Expected output:

```
pgvector extension ready
All migrations complete!
```

TIP: If you get a connection error, double-check your DATABASE_URL in .env. Make sure it includes ?sslmode=require for Neon.

5. Phase 2 — Web Scraping & Ingestion

Build a scraper that crawls admissions.byuh.edu, chunks the text, generates embeddings, and stores everything in the database.

5.1 Build the Web Scraper

Write a web scraper that crawls https://admissions.byuh.edu/ using BFS. Use axios and cheerio. Stay within the admissions.byuh.edu hostname. Strip nav, footer, header, scripts, styles. Limit to 100 pages. Add a 500ms delay between requests.

Create src/lib/scraper.ts

Key features of the scraper:

- BFS (breadth-first search) crawl starting from the homepage
- Stays within admissions.byuh.edu — never follows external links
- Strips non-content elements: scripts, styles, nav, footer, iframes
- Prefers semantic containers: <main>, <article>, [role=main]
- Normalizes URLs (removes trailing slashes, fragments, query params)
- Skips non-HTML resources (PDFs, images, CSS, JS files)
- Rate limits at 500ms between requests
- Caps at 100 pages maximum

5.2 Build the Text Chunker

Create a text chunking function with chunk size 1000 characters and 200 character overlap using a sliding window approach.

Create src/lib/chunker.ts:

```
export function chunkText(
  text: string,
  chunkSize = 1000,
  overlap = 200
): string[] {
  const chunks: string[] = [];
  let start = 0;

  while (start < text.length) {
    const end = Math.min(
      start + chunkSize, text.length
    );
    chunks.push(text.slice(start, end));
    if (end === text.length) break;
    start += chunkSize - overlap;
  }

  return chunks;
}
```

Why 1000/200? Each chunk is ~250 tokens. The 200-char overlap ensures context isn't lost at boundaries. This is a solid default for most RAG systems.

5.3 Build the Ingestion Pipeline

Create an ingestion script that scrapes admissions.byuh.edu, chunks the content, generates embeddings using Vercel AI Gateway with OpenAI text-embedding-3-small, and stores everything in the byuh_chunks table. Use p-limit for concurrency control (5 concurrent embedding requests).

Create src/scripts/ingest.ts

This script does everything in one pass:

1. Calls the scraper to crawl admissions.byuh.edu
2. For each scraped page, chunks the text content
3. Deletes existing chunks for that URL (idempotent re-runs)
4. Generates embeddings via the AI Gateway (5 concurrent)
5. Inserts each chunk + embedding into byuh_chunks

5.4 Run the Ingestion

```
pnpm ingest
```

Expected output (takes a few minutes):

```
Starting ingestion pipeline...
Scraping: https://admissions.byuh.edu/
  -> BYU-Hawaii Admissions (4523 chars)
Scraping: https://admissions.byuh.edu/apply
  -> Apply to BYU-Hawaii (3891 chars)
...
Crawl complete: 47 pages scraped
Processing ... -> 6 chunks
......
Ingestion complete!
```

TIP: Safe to re-run anytime. It deletes old chunks per URL before re-inserting, so you can use it to refresh after site updates.

6. Phase 3 — Chat API (RAG + LLM)

Build the API endpoint that powers the chatbot. This is where the RAG magic happens.

6.1 How the Chat API Works

When a user sends a message, the API:

1. Extracts the latest user message from the request
2. Generates an embedding of the question (same model as chunks)
3. Runs cosine similarity search via pgvector's <=> operator
4. Retrieves the top 5 most relevant chunks
5. Builds a system prompt with the retrieved context
6. Streams the LLM response back to the client

6.2 Create the Chat Route

Build a POST /api/chat route that: (1) extracts user message from UIMessage format, (2) saves user message to DB, (3) auto-titles conversations, (4) embeds query with text-embedding-3-small via AI gateway, (5) does cosine similarity search on byuh_chunks, (6) builds system prompt with context, (7) streams response with gpt-4o-mini, (8) saves assistant response on finish.

Create src/app/api/chat/route.ts

Key implementation details:

- **Vector search:** pgvector <=> operator (cosine distance) to find 5 most relevant chunks
- **Context injection:** Each chunk prefixed with source URL so the LLM can cite sources
- **Streaming:** streamText() from AI SDK for real-time token-by-token responses
- **Persistence:** Saves both user and assistant messages to chat_messages table
- **Auto-titling:** First message automatically becomes the conversation title

The system prompt instructs the LLM to:

- Only answer using the provided context
- Be warm and encouraging like an admissions counselor
- Suggest follow-ups naturally at the end of each response
- Admit when information isn't in the context

6.3 Session Management

Create a session helper that uses HTTP-only cookies to track anonymous visitors. Create or retrieve a session from the sessions table. 1-year TTL.

Create src/lib/session.ts

This module:

- Checks for an existing byuh_session cookie

- Verifies the session exists in the database
- Creates a new session + sets a 1-year HTTP-only cookie if needed
- Used by all API routes to scope conversations to a visitor

6.4 Conversation API Routes

Create three additional API routes:

- **src/app/api/conversations/route.ts** — List (GET) and Create (POST)
- **src/app/api/conversations/[id]/route.ts** — Delete (DELETE)
- **src/app/api/messages/route.ts** — Load messages for a conversation (GET)

7. Phase 4 — Frontend (Chat UI)

Build the user-facing chat interface.

7.1 Layout & Styling

Set up the root layout with Montserrat font from Google Fonts. Create a globals.css with BYUH brand colors (crimson #BA0C2F, burgundy #9E1B34, gold #946F0A, navy #002E5D) using Tailwind CSS v4 @theme syntax. Add @tailwindcss/typography plugin.

BYUH Brand Palette:

Color	Hex	Usage
Crimson	#BA0C2F	Primary buttons, header, user messages
Burgundy	#9E1B34	Sidebar background, hover states
Gold	#946F0A	Links in assistant responses
Navy	#002E5D	Secondary accents

7.2 Build the Chat UI

Build a ChatGPT-style chat UI with: (1) sidebar showing conversation history with create/delete, (2) BYUH crimson header bar, (3) message bubbles with markdown rendering, (4) welcome screen with suggestion chips, (5) typing indicator, (6) mobile-responsive with slide-in sidebar. Use useChat from @ai-sdk/react with DefaultChatTransport.

Create src/app/page.tsx

Key UI features:

- **Sidebar:** Lists conversations with active state, new chat + delete buttons
- **Messages:** User = crimson bubbles, Assistant = white cards with markdown via react-markdown + remark-gfm
- **Welcome screen:** BYUH logo, intro text, 4 suggestion buttons
- **Typing indicator:** Three pulsing dots while waiting for response
- **Mobile:** Sidebar slides in as overlay on small screens
- **Auto-scroll:** Scrolls to latest message automatically

NOTE: *AI SDK v6 changed the useChat API. Use DefaultChatTransport (not TextStreamChatTransport) and UIMessage with .parts[] for content. API must return toUIMessageStreamResponse().*

8. Running the Application

Complete sequence from zero to running chatbot:

8.1 First-Time Setup

1. Clone the repository:

```
git clone <your-repo-url>
cd byuh-admissions-bot
```

2. Install dependencies:

```
pnpm install
```

3. Create .env file with DATABASE_URL and AI_GATEWAY_API_KEY

4. Run database migrations:

```
pnpm db:migrate
```

5. Run the ingestion pipeline (scrape + embed):

```
pnpm ingest
```

6. Start the development server:

```
pnpm dev
```

7. Open **http://localhost:3000** in your browser

8.2 Refreshing Content

When the admissions website changes, re-run:

```
pnpm ingest
```

It re-scrapes, re-chunks, and re-embeds. Deletes old chunks per URL first, so it’s safe to repeat.

9. Project Structure

```
byuh-admissions-bot/
├── .env                    # API keys (never commit)
├── package.json
├── tsconfig.json
├── drizzle.config.ts       # Drizzle Kit config
├── next.config.ts
├── public/
│   └── logo.png            # BYUH logo
├── src/
│   ├── app/
│   │   ├── layout.tsx      # Root layout + font
│   │   ├── page.tsx        # Chat UI
│   │   ├── globals.css     # Brand palette
│   │   └── api/
│   │       ├── chat/
│   │       │   └── route.ts  # RAG endpoint
│   │       ├── conversations/
│   │       │   ├── route.ts  # List + create
│   │       │   └── [id]/
│   │       │       └── route.ts # Delete
│   │       └── messages/
│   │           └── route.ts  # Load messages
│   ├── db/
│   │   ├── index.ts        # DB connection
│   │   ├── schema.ts       # Table definitions
│   │   └── migrate.ts      # SQL migrations
│   ├── lib/
│   │   ├── scraper.ts      # BFS web crawler
│   │   ├── chunker.ts      # Text splitter
│   │   └── session.ts      # Cookie sessions
│   └── scripts/
│       └── ingest.ts       # Full pipeline
```

10. Cost Breakdown

Service	Cost	Notes
Vercel hosting	$20/mo (Pro)	Free Hobby tier for testing
Neon database	Free tier	0.5 GB storage
OpenAI embeddings	~$0.02 one-time	~300 chunks
GPT-4o-mini chat	~$0.01/convo	Very cheap per-token

Total: ~$20/month with near-zero per-query costs.

11. Troubleshooting

"No database connection string was provided"

- Check your .env file exists and DATABASE_URL is set
- dotenv is loaded automatically in db/index.ts

"CREATE INDEX CONCURRENTLY cannot run inside a transaction"

- Use ivfflat index (as in this SOP), not HNSW with CONCURRENTLY
- Or run the index creation manually outside of migrations

"Cannot find module 'ai/react'"

- AI SDK v6 moved useChat to @ai-sdk/react
- Run: pnpm add @ai-sdk/react

"toDataStreamResponse is not a function"

- Use toUIMessageStreamResponse() in AI SDK v6

Embeddings are slow or timing out

- Use p-limit to control concurrency (5 is a safe default)
- Reduce concurrency if you see 429 rate limit errors

Useful Links

- **AI SDK:** ai-sdk.dev/docs
- **Drizzle:** orm.drizzle.team
- **Neon:** neon.tech/docs
- **pgvector:** github.com/pgvector/pgvector
- **Claude Code:** claude.com/claude-code

12. Adapting for Other Websites

This architecture works for any website. Here's what to change:

1. **Scraper:** Change BASE_URL and hostname check. Adjust content selectors for different HTML structures.
2. **Chunk size:** 1000/200 is a good default. Use 1500/300 for long docs, 500/100 for short FAQs.
3. **Embedding model:** text-embedding-3-small (1536d) for most cases. For higher accuracy, text-embedding-3-large (3072d) — update vector() in schema.
4. **LLM:** GPT-4o-mini is fast and cheap. Upgrade to GPT-4o or Claude for more nuanced answers. Just change the model string.
5. **System prompt:** Rewrite to match your domain. Tone, guardrails, and instructions should reflect the assistant you want.
6. **UI:** Update colors in globals.css, swap logo in public/, adjust header text and suggestion chips.

Appendix: AI Prompt Reference

These are the exact prompts used with Claude Code to build each part of this project. Copy and modify for your needs.

We have a basic Next.js application and I want you to install Drizzle ORM and set it up. We also have the database connection string in the env.

We're building a RAG chatbot so I want you to build a schema for the chunks and embeddings. We'll scrape the https://admissions.byuh.edu/ for content and store the embeddings.

Now it's time for scraping the site that I told you about. I want you to write an ingestion script to scrape the content and chunk with chunk sizes of 1000 and overlap of 200.

I want you to create the embedding pipeline. I want to use Vercel's AI SDK and AI Gateway and I already have the AI Gateway key in my env.

We now have the chunks and embeddings. Let's build the chat API. Use the AI SDK to generate text and do a cosine similarity search.

Create a minimal frontend for chatbot UI. Use the useChat() hook to talk to the chat API that you just created.

Add markdown rendering for AI responses with react-markdown and remark-gfm. Make the UI mobile-friendly.

Add conversation history with session cookies and a ChatGPT-style sidebar UI. Include create/delete conversations, auto-titling from the first message, and BYUH branding.

End of SOP